AYE
TECH HUB

• INDUSTRIAL ROBOTICS • LESSON 03

Robot Kinematics and Coordinate Systems

How the Robot Knows Exactly Where Its Tool Is

Lesson 03 of 30 | Industrial Robotics Program

Awet G. Nway

Founder, AYE Tech Hub | Industrial Robotics & Automation Specialist

- Forward kinematics: calculate TCP position from joint angles
- Inverse kinematics: calculate joint angles from desired TCP position
- The 5 robot coordinate systems every programmer must know
- Joint space vs Cartesian space — when to use each
- Singularities: what they are and how to avoid them

awetgknway@gmail.com

What Is Robot Kinematics?

Kinematics is the study of motion without considering the forces that cause it. For robots, kinematics answers one fundamental question: **given the joint angles, where is the tool — and given where you want the tool, what must the joint angles be?** Every robot movement in every program is computed by solving these two problems thousands of times per second.

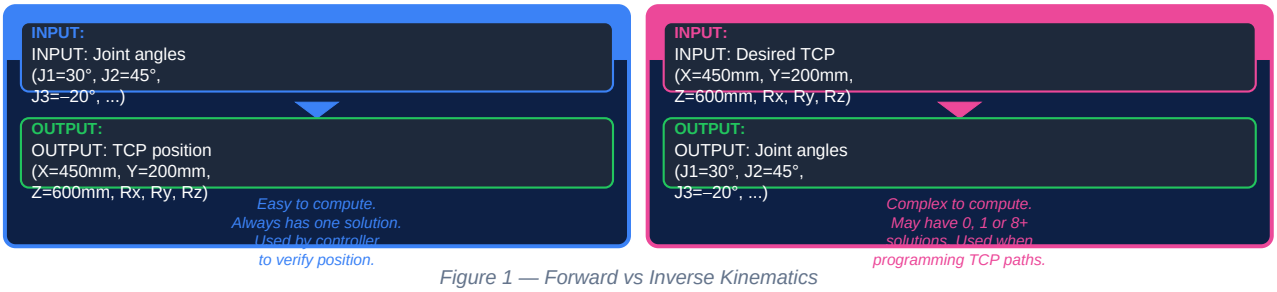

Figure 1 — Forward vs Inverse Kinematics

Problem	Mathematical Name	Difficulty	Robot Use
Given joint angles → find TCP pose	Forward Kinematics (FK)	Easy — one unique answer. Multiplication of transformation matrices.	Controller verification, simulation, monitoring actual TCP position
Given desired TCP pose → find joint angles	Inverse Kinematics (IK)	Hard — may have 0, 2, 4, or 8 solutions for 6-DOF robots.	All TCP-based programming (MoveL, MoveC, path planning)
Move from A to B avoiding obstacles	Motion Planning / Path Planning	Complex — collision detection, joint limits, singularity avoidance.	Offline programming, simulation, collaborative robot safety
Compute velocities and accelerations	Differential Kinematics (Jacobian)	Moderate — Jacobian matrix maps joint velocities to TCP velocities.	Force/torque control, compliance, speed-limited paths

Why does IK have multiple solutions? For a 6-DOF robot, reaching the same TCP position and orientation can often be done in several different arm configurations: "elbow up" vs "elbow down", "wrist flip" vs "no wrist flip", "forward" vs "backward" arm. The robot controller must choose the best configuration — usually the one closest to the current position.

Robot Coordinate Systems — The 5 Frames

Every robot manufacturer defines a set of **coordinate frames** that describe where things are in space. Understanding these frames is essential for programming — the same TCP position means completely different things depending on which frame you are using.

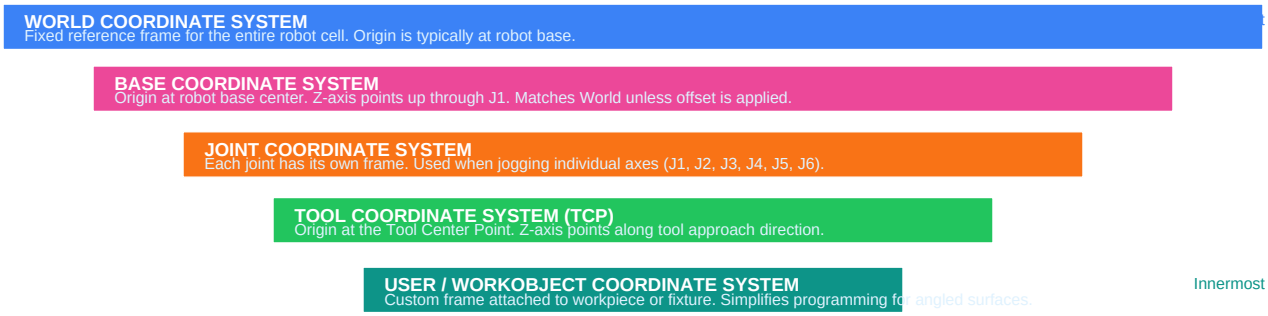

Figure 2 — Robot Coordinate System Hierarchy (World → Base → Joint → TCP → User)

Frame	ABB Name	KUKA Name	Fanuc Name	When to Use
World Frame	World	WORLD	$WORLD	Multi-robot cells; external axes; when absolute position matters
Base Frame	Base	ROBROOT	$UFRAME (=world)	Default frame. All positions are relative to robot base unless changed.
Joint Frame	Axis (J1–J6)	AXIS / A1–A6	J1–J6	Jogging individual joints; setting home position; avoiding singularities
Tool Frame (TCP)	Tool	TOOL	$TOOL	Any TCP motion (MoveL, MoveC); defining orientation of tool approach
User/Work Frame	Wobj (WorkObject)	BASE	$UFRAME	Workpiece-relative programming; angled fixtures; conveyor tracking

Practical Scenario	Which Frame to Use	Why
Jogging robot manually during setup	Joint frame (Axis/J mode)	Most intuitive — each button moves one joint predictably
Teaching a point on the workpiece	User/Work frame aligned to workpiece	If fixture moves, just update the workobject — all points update too
Running a straight weld path	Tool frame (TCP) with Cartesian motion	MoveL keeps torch perpendicular and moves in straight line to weld point
Recovering from an alarm near robot limits	Joint frame — move joint by joint	Avoids unexpected Cartesian motion that could cause further collision
Programming an arc motion (MoveC)	Tool frame (Cartesian)	MoveC requires TCP to follow a circular arc — only possible in Cartesian mode

Joint Space vs Cartesian Space and Singularities

When you command a robot to move from Point A to Point B, the controller must decide **how** the tool gets there. The two fundamental motion types — Joint Space (MoveJ) and Cartesian Space (MoveL) — produce completely different tool paths and have different speed, safety, and programming implications.

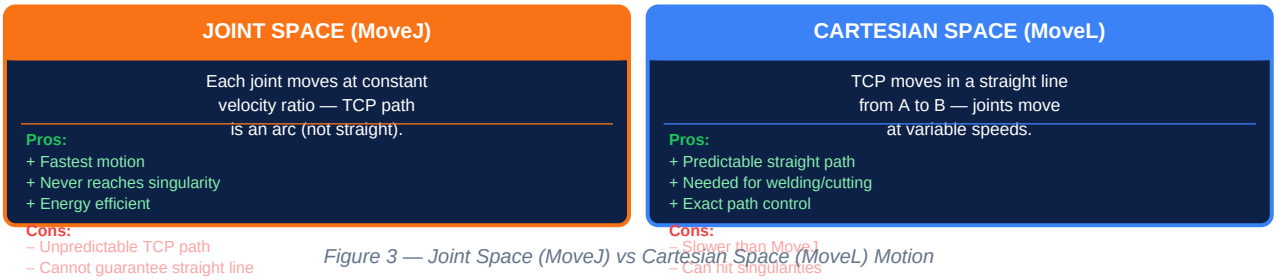

Figure 3 — Joint Space (MoveJ) vs Cartesian Space (MoveL) Motion

Motion Type	ABB	KUKA	Fanuc	Path Shape	Typical Use
Joint (Point-to-Point)	MoveJ	PTP	J	Arc / undefined	Free moves, repositioning, large moves
Linear (Cartesian)	MoveL	LIN	L	Straight line	Welding, cutting, assembly paths, gluing
Circular	MoveC	CIRC	C	Arc	Circular welds, curved paths, dispensing
Spline	MoveAbsJ	SPLINE	—	Smooth curve	Complex curved paths with smooth blending

Singularities — What They Are and How to Avoid Them

Singularity Type	When It Occurs	Effect	Prevention
Shoulder / Overhead Singularity	TCP is directly above the robot base. J1 must rotate infinitely fast.	Robot slows or stops. Cannot pass through this zone.	Program paths to avoid the overhead position. Use MoveJ to jump over it.
Elbow Singularity	J3 is fully extended or fully folded — arm is fully straight or fully bent.	Large joint accelerations required. Robot may alarm or slow dramatically.	Keep elbow slightly bent. Use a non-singularity arm configuration.
Wrist Singularity (most common)	J4 and J6 become coaxial (J5 ≈ 0°). Infinite solutions — robot "flips" wrist.	Joints J4 and J6 rotate very fast or flip 180°. Robot alarms at speed limit.	Keep J5 > 5° from zero. Use SingArea Wrist or equivalent setting in ABB/KUKA.

TCP Position Notation	Meaning	Example (ABB Rapid)
X, Y, Z	Position of TCP in mm relative to active frame	trans:=[450, 200, 600]
Rx, Ry, Rz (Euler angles)	Orientation as rotation around X, Y, Z axes	rot:=[0.707, 0, 0.707, 0] (quaternion)
Quaternion (q1, q2, q3, q4)	More reliable than Euler — no gimbal lock	Most modern controllers use quaternions internally
Robot Configuration (cf1, cf4, cf6, cfx)	Which of the 8 IK solutions to use	conf:=[0, 0, 0, 0] = elbow up, wrist no-flip, forward
External Axes (eax_a...eax_f)	Position of positioners, tracks, turntables	extax:=[9E9, 9E9, 9E9, 9E9, 9E9, 9E9] = not used

What comes next — ROBOT-004: ABB Robot Systems and Programming — ABB robot families (IRB series), the RobotStudio simulation environment, RAPID programming language basics, creating your first MoveL and MoveJ program, and managing I/O signals.